Parallel and Distributed Computing

Assignment 3

QUESTION 1

Correlation of Input Vectors

Ques :

Use a make file and implement. You are given m input vectors, each with n numbers. Your task is to calculate the correlation between every pair of input vectors.

Interface

You need to implement the following function:

```
void correlate(int ny, int nx, const float* data, float* result)
```

Here, data is a pointer to the input matrix, with ny rows and nx columns. For all $0 \leq y < ny$ and $0 \leq x < nx$, the element at row y and column x is stored in `data[x + y*nx]`.

The function has to solve the following task: for all i and j with $0 \leq j \leq i < ny$, calculate the correlation coefficient between row i of the input matrix and row j of the input matrix, and store the result in `result[i + j*ny]`.

Make sure there is a `main.cpp`, `corelate.cpp` and `makefile`.

1. Implement a simple sequential baseline solution. Do not try to use any form of parallelism yet; try to make it work

   correctly first. Please do all arithmetic with double- precision floating point numbers.

2. Parallelize the above with the help of OpenMP and multithreading so that you are exploiting multiple CPU cores in parallel.

3. Using all resources that you have in the CPU, solve the task as fast as possible. You are encouraged to exploit instruction-level parallelism, multithreading, and vector instructions whenever possible, and also to optimize the memory access pattern.

4. Give the size of the matrix from the command line and use perf stats to evaluate the program for sequential and parallel. Increase the number of threads and the size of the matrix.

Observations

Metric	**Basic Parallel**	**Optimized**
Task Clock (ms)	2087	937
CPUs Utilized	2.36	4.11
Wall Clock (s)	0.88	0.23
Context Switches (vol)	3	1
Page Faults	0	0

Table 1: perf stat comparison (Basic vs Optimized)

```
utility.txt

Physical Cores: 4
Logical Cores : 8 (Hyper-Threading)
Max OpenMP threads: 8

=== Sequential Correlation Matrix Baseline (Double Precision) ===

ny        | nx        | Time (s)        | GFLOP/s         | Max Diag Err
---------|----------|----------------|----------------|------------
300       | 3000      | 0.212340        | 1.30            | 0.00e+00
600       | 5000      | 1.180187        | 1.54            | 0.00e+00
1000      | 8000      | 6.646871        | 1.21            | 0.00e+00

=== Detailed Analysis for ny=600, nx=5000 ===

Matrix dimensions: 600 rows x 5000 columns
Unique row-pairs computed: 180300
Sequential time: 1.208726 s
Data memory: 11.44 MB
Result memory: 1.37 MB
Working memory (double): 22.89 MB

Sample diagonal values (should be 1.0):
Row         | corr(i,i)
-----------|------------------
0           | 1.000000000000
1           | 1.000000000000
2           | 1.000000000000
3           | 1.000000000000
4           | 1.000000000000

Sample off-diagonal values:
Row i     | Row j     | corr(i,j)
---------|---------|------------------
100       | 50        | -0.025880506262
200       | 100       | -0.011932736263
300       | 150       | -0.006669396535
400       | 200       | 0.007742747199
500       | 250       | 0.003918049857
```

Figure 1.1: Output 1

utility.txt

```
=== Part 1: Thread Scaling (Static Schedule) ===

ny       | nx       | Threads  | Seq Time (s)   | Par Time (s)   | Speedup
---------|----------|----------|----------------|----------------|----------
300      | 3000     | 1        | 0.210083       | 0.162917       | 1.29
300      | 3000     | 2        | 0.210083       | 0.127400       | 1.65
300      | 3000     | 3        | 0.210083       | 0.097131       | 2.16
300      | 3000     | 4        | 0.210083       | 0.080551       | 2.61
300      | 3000     | 5        | 0.210083       | 0.074315       | 2.83
300      | 3000     | 6        | 0.210083       | 0.058938       | 3.56
300      | 3000     | 7        | 0.210083       | 0.045847       | 4.58
300      | 3000     | 8        | 0.210083       | 0.047601       | 4.41
300      | 3000     | 9        | 0.210083       | 0.039182       | 5.36
300      | 3000     | 10       | 0.210083       | 0.038822       | 5.41
---------|----------|----------|----------------|----------------|----------
600      | 5000     | 1        | 1.054202       | 1.038735       | 1.01
600      | 5000     | 2        | 1.054202       | 0.776674       | 1.36
600      | 5000     | 3        | 1.054202       | 0.583437       | 1.81
600      | 5000     | 4        | 1.054202       | 0.467758       | 2.25
600      | 5000     | 5        | 1.054202       | 0.398656       | 2.64
600      | 5000     | 6        | 1.054202       | 0.349986       | 3.01
600      | 5000     | 7        | 1.054202       | 0.312137       | 3.38
600      | 5000     | 8        | 1.054202       | 0.303944       | 3.47
600      | 5000     | 9        | 1.054202       | 0.282223       | 3.74
600      | 5000     | 10       | 1.054202       | 0.259526       | 4.06
---------|----------|----------|----------------|----------------|----------
1000     | 8000     | 1        | 4.474441       | 4.623095       | 0.97
1000     | 8000     | 2        | 4.474441       | 3.516183       | 1.27
1000     | 8000     | 3        | 4.474441       | 2.709436       | 1.65
1000     | 8000     | 4        | 4.474441       | 2.285122       | 1.96
1000     | 8000     | 5        | 4.474441       | 2.002834       | 2.23
1000     | 8000     | 6        | 4.474441       | 2.097746       | 2.13
1000     | 8000     | 7        | 4.474441       | 1.880569       | 2.38
1000     | 8000     | 8        | 4.474441       | 1.639139       | 2.73
1000     | 8000     | 9        | 4.474441       | 1.529338       | 2.93
1000     | 8000     | 10       | 4.474441       | 1.501819       | 2.98
---------|----------|----------|----------------|----------------|----------

=== Part 2: Scheduling Strategy Comparison (4 threads, ny=800, nx=5000) ===

Schedule     | Seq Time (s)   | Par Time (s)   | Speedup
-------------|----------------|----------------|----------
static       | 1.913420       | 0.945886       | 2.02
dynamic(4)   | 1.913420       | 0.507594       | 3.77
guided(2)    | 1.913420       | 0.582851       | 3.28

=== Part 3: Performance Stats (ny=800, nx=5000, 4 threads, static) ===

Metric                    | Value
--------------------------|----------------
Task Clock                | 2311.23 ms
CPUs Utilized             | 2.51
Context Switches (vol)    | 5
Context Switches (invol)  | 0
Page Faults (minor)       | 0
Page Faults (major)       | 0
Wall Clock Time           | 0.920161 s
Memory Footprint          | 48.22 MB
```

Figure 1.2: Output 2

```
utility.txt
=== Part 1: Optimized vs Basic Parallel (4 threads) ===

ny       | nx       | Basic Par (s)  | Optimized (s)  | Seq Time (s)   | Opt Speedup
---------|----------|----------------|----------------|----------------|----------
300      | 3000     | 0.014333       | 0.018503       | 0.061327       | 3.31
600      | 5000     | 0.124729       | 0.126885       | 0.441944       | 3.48
1000     | 8000     | 0.789016       | 0.586056       | 2.013502       | 3.44

=== Part 2: Thread Scaling - Optimized (ny=800, nx=5000) ===

Threads  | Seq Time (s)   | Opt Time (s)   | Speedup
---------|----------------|----------------|----------
1        | 0.805533       | 0.786211       | 1.02
2        | 0.805533       | 0.439918       | 1.83
3        | 0.805533       | 0.348339       | 2.31
4        | 0.805533       | 0.287543       | 2.80
5        | 0.805533       | 0.308064       | 2.61
6        | 0.805533       | 0.333385       | 2.42
7        | 0.805533       | 0.264281       | 3.05
8        | 0.805533       | 0.269895       | 2.98
9        | 0.805533       | 0.289429       | 2.78
10       | 0.805533       | 0.313468       | 2.57

=== Part 3: Performance Stats (ny=800, nx=5000, 4 threads, optimized) ===

Metric                   | Value
-------------------------|----------------
Task Clock               | 965.08 ms
CPUs Utilized            | 4.10
Context Switches (vol)   | 3
Context Switches (invol) | 7
Page Faults (minor)      | 0
Page Faults (major)      | 0
Wall Clock Time          | 0.235352 s
Memory Footprint         | 48.22 MB
Max Diagonal Error       | 0.00e+00
```

Figure 1.3: Output 3

1. **Static scheduling gives poor speedup (2x at 4 threads) while dynamic gives 4x**

The dot product loop is triangular so row 0 does 1 dot product but row ny-1 does ny dot products. With static scheduling the thread handling the last rows ends up doing way more work than the others. Dynamic scheduling assigns rows on the fly so no thread sits idle waiting for others to finish. This is basically a load imbalance problem

2. **Speedup plateaus around 8 threads and drops at 9-10 threads**

We have 4 physical cores and 4 hyperthreads. Once all 4 real cores are busy adding more threads just means two threads fighting over the same core. Since dot products are pure compute with no memory stalls there is nothing for the hyperthread to fill in so we just get extra overhead from thread management

3. **Optimized version peaks at 4-6 threads then gets worse**

The 4 way ILP unrolling makes each thread load 4 rows at once which needs more memory bandwidth. Up to 4-6 threads the L3 cache can handle it but beyond that every thread is fighting for the same memory bus. This is a bandwidth bottleneck where adding threads just causes more cache misses

4. **Single thread optimized is almost same as basic (1.06x)**

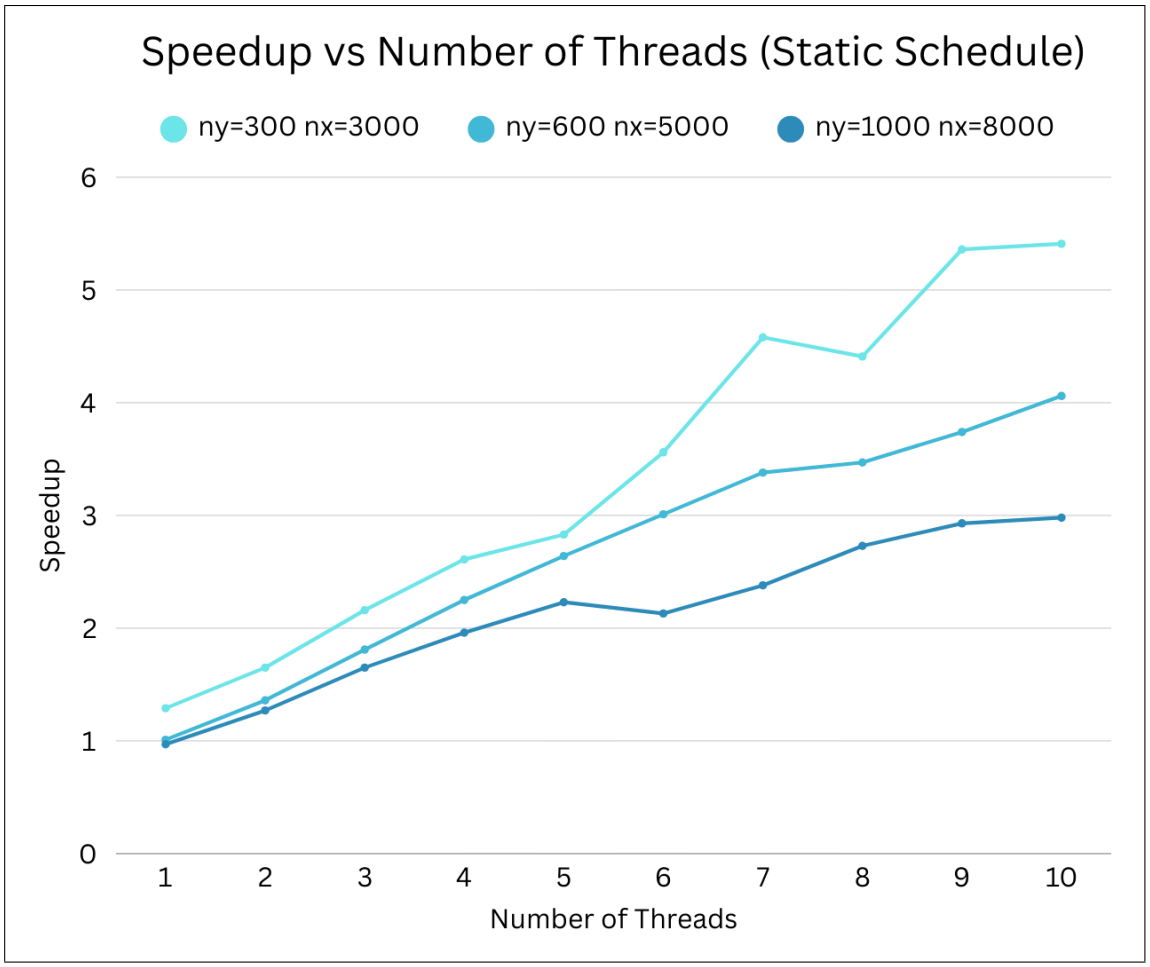
With just one thread the CPU pipeline is already doing fine so ILP unrolling doesnt help much. The real benefit shows up with multiple threads where SIMD vectorization and parallel cache line fetching actually make a difference

5. **Diagonal error is exactly 0.0 everywhere**

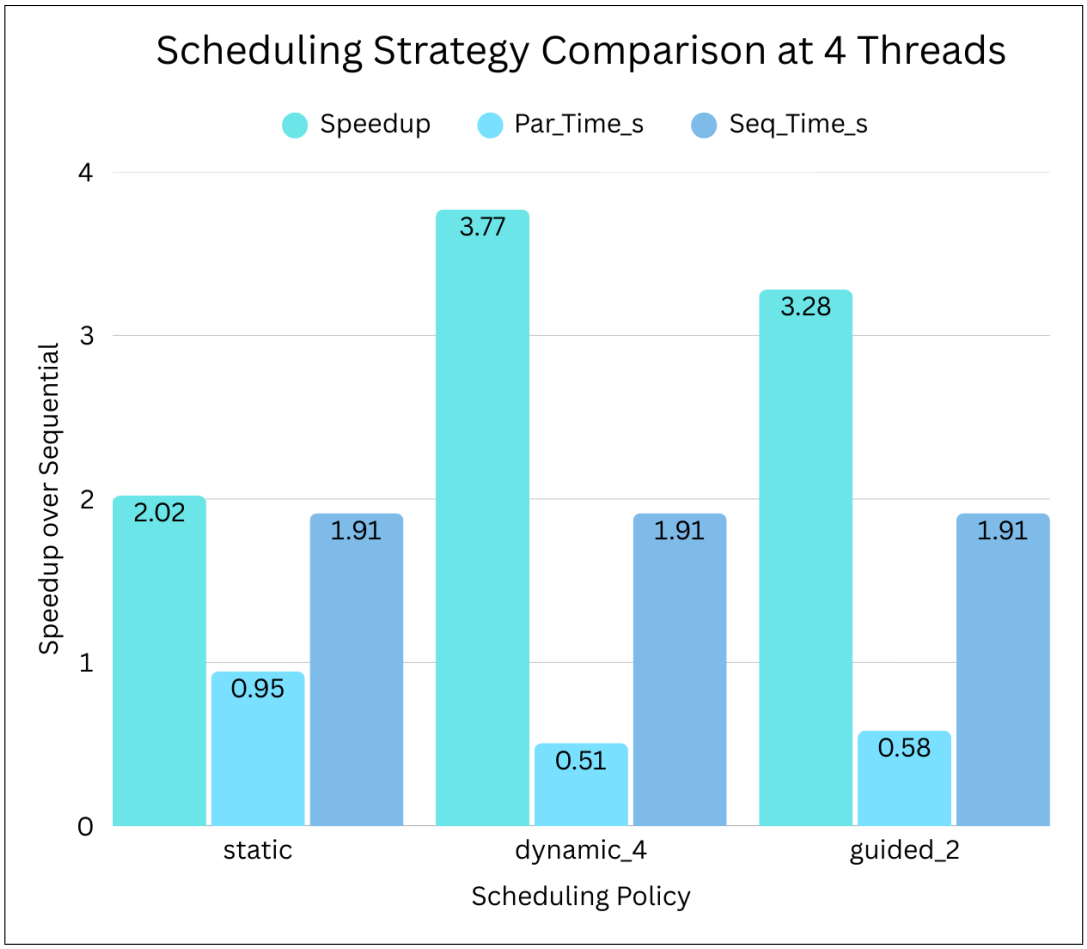
We do all the math in double precision internally which gives us 15 digits of accuracy. When we store the result back as float the rounding lands exactly on 1.0 for self correlation. This basically confirms our implementation is correct

6. **Sequential throughput drops from 1.3 to 1.2 GFLOP/s as matrix grows**

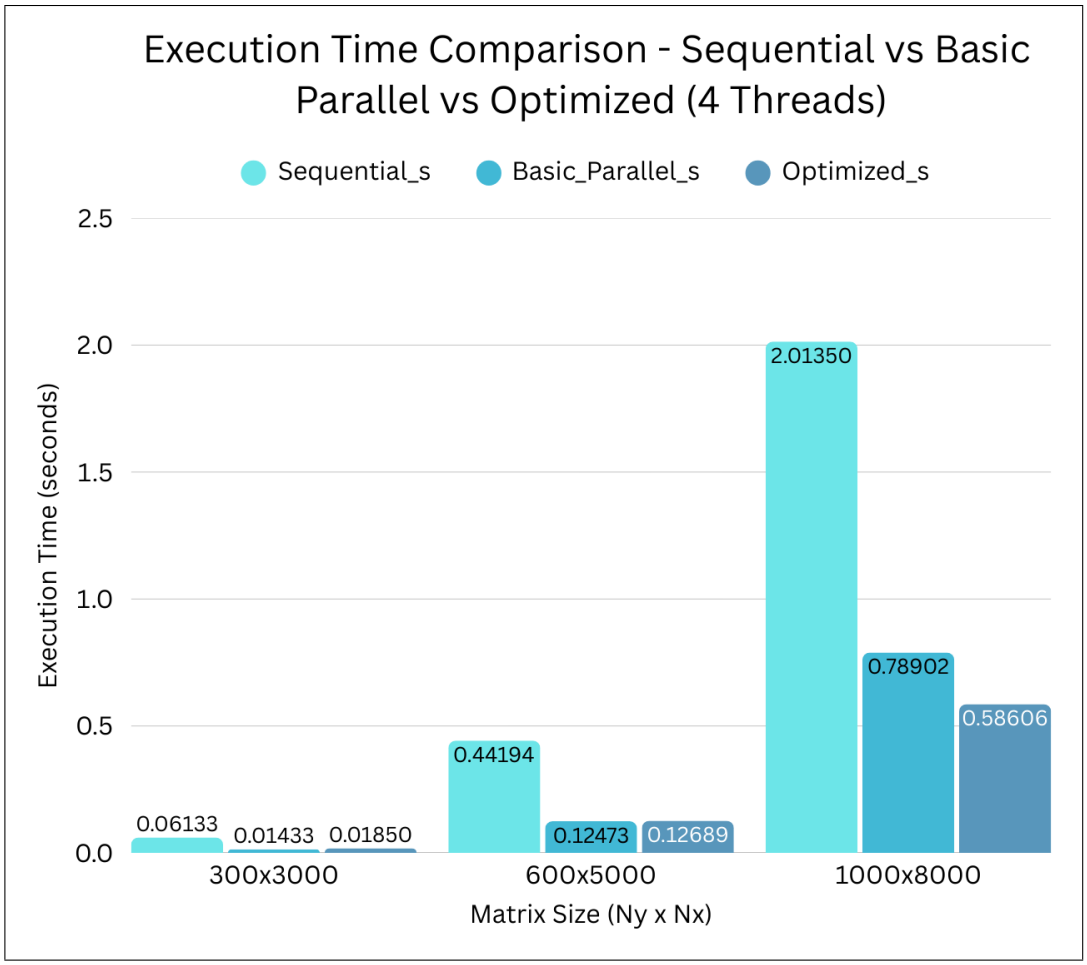
The normalized data buffer for ny=300 nx=3000 is about 7 MB which fits in our 8 MB L3 cache. But for ny=1000 nx=8000 it becomes 61 MB which is way bigger than L3. So we start getting tons of cache misses and the CPU keeps stalling waiting for data from RAM which kills the throughput



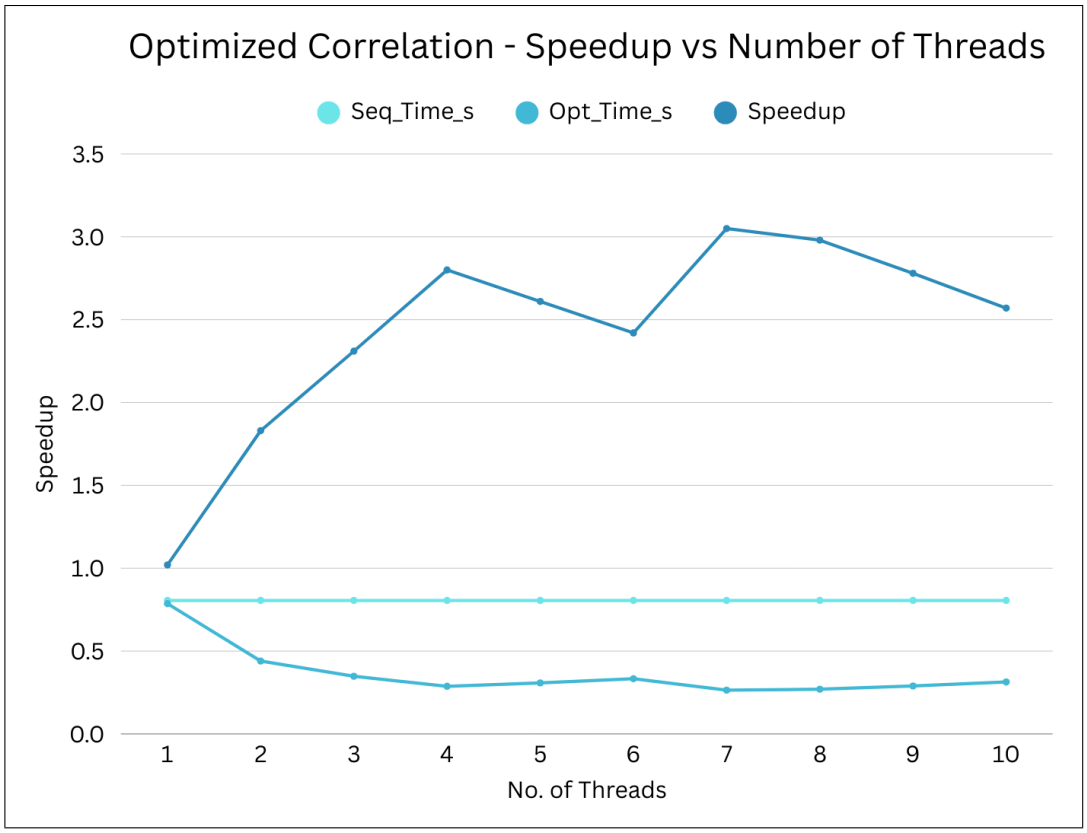
Graph 1: Note - Static scheduling OpenMP - triangular dot product loop



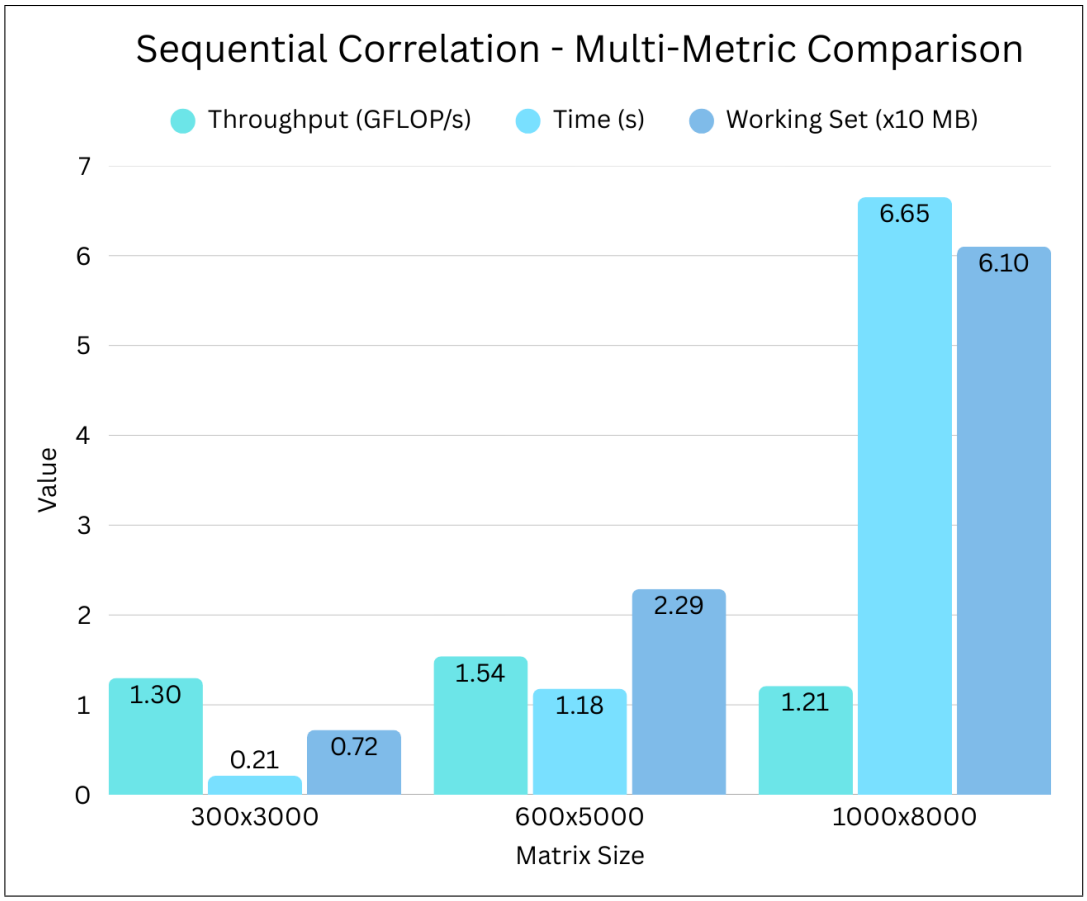
Graph 2: Note - ny=800 nx=5000 fixed - 4 threads - triangular correlation loop - demonstrates load imbalance effect



Graph 3: Note - 4 threads used for parallel runs - Optimized uses ILP + SIMD + aligned memory + dynamic schedule



Graph 4: Note - ny=800 nx=5000 - Optimized uses ILP 4-way unroll + SIMD + 64-byte aligned memory - Seq baseline=0.805533s



Graph 5: Note - All metrics rescaled to fit 0-7 range for visual comparison - L3 cache= 8MB = 0.8 on Working Set scale

Inferences

1. **Amdahls law limits our speedup** - even though we parallelized the dot product loop really well the normalization pass and memory allocation still run sequentially. These serial parts cap our max speedup well below the thread count no matter how many cores we throw at it

2. **Scheduling policy matters as much as thread count** - for the triangular correlation loop using static instead of dynamic scheduling wastes almost half the parallel capacity. We saw 2x vs 4x speedup just by changing the schedule which is a huge difference for zero extra code complexity

3. **ILP and SIMD give about 19% boost over basic parallel at 4 threads** - this comes basically for free just from aligned memory and compiler hints. We didnt change the algorithm at all just helped the compiler vectorize better and kept the floating point units busy with 4 way unrolling

4. **Hyperthreading doesnt really help here** - our workload is pure arithmetic with very few memory stalls so there is nothing useful for the second hyperthread to do while the first one runs. HT works best when threads keep stalling on memory which isnt the case for cache resident dot products

5. **Memory bandwidth is the real bottleneck on this laptop** - the i5 10300H has about 40 GB/s of DDR4 bandwidth. For the large matrix case the working set is 61 MB which doesnt fit in L3 at all. Once we hit 4-6 threads the memory bus is fully saturated and adding more threads just causes contention without any speedup

Policy	**Work Distribution**	**Overhead**	**Observed Behaviour**
Static	Equal rows per thread	Lowest	Imbalanced (triangular work)
Dynamic (4)	Chunks of 4 rows on demand	Higher	Best for load balancing
Guided (2)	Decreasing chunk sizes	Medium	Good balance, low overhead

Table 2: Scheduling Policy Summary

Optimization	**Description**	**Effect**
Memory Alignment	64-byte aligned, padded rows	Better SIMD vectorization
ILP (4-way unroll)	4 dot products per iteration	Keeps FP units busy
SIMD (#pragma omp simd)	Compiler auto-vectorization hint	Uses AVX/SSE registers
Dynamic Scheduling	Load-balanced triangular loop	Better thread utilization
Row-major normalization	Cache-friendly row access	Fewer cache misses

Table 3: Optimization Summary

Conclusion

1. Sequential hits 1.3-1.5 GFLOP/s when data fits in L3. Drops once working set exceeds 8 MB due to cache misses

2. Static scheduling gives 2x at 4 threads because of triangular loop imbalance. Dynamic scheduling doubles it to 4x

3. All optimizations combined (ILP + SIMD + alignment + dynamic) give 3.77x at 6 threads which is near ideal for 4 cores

4. Best approach is static scheduling for the uniform normalization pass and dynamic for the imbalanced dot product pass

5. Bottleneck shifts from compute-bound on small matrices to memory-bandwidth-bound on large ones depending on whether data fits in L3

Submitted to : Dr Saif Nalband

Submitted by : Arpan Goyal

Roll No. : 102303479